

Final Year Project Report

Half Unit – Final Report

Timetabling Optimisation via Metaheuristic Algorithms

William Green

A report submitted in part fulfilment of the degree of

BSc Computer Science and Mathematics

Supervisor: Li Zhang



Department of Computer Science
Royal Holloway, University of London

January 26, 2026

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 9063

Student Name: William Green

Date of Submission: 26.01.2026

Signature:

A handwritten signature in black ink, appearing to read 'W Green', written in a cursive style.

Table of Contents

Abstract

Personal Motivation and Future Plans

Theoretical Background and Useful Definitions

Chapter 1: Prep-work and Baseline Algorithms

1.1 Preliminary Work

1.2 Brute Force

1.3 Nearest Neighbour

Chapter 2: Metaheuristics on the Travelling Salesman Problem

2.1 Simulated Annealing

2.2 Particle Swarm Optimisation

2.3 Genetic Algorithm

2.4 Combined Analysis of Metaheuristics on the TSP

Chapter 3: Adapting Metaheuristics to the Timetabling Problem

3.1 Timetabling Problem Representation

3.1.1 Design Choices

3.1.2 Coding a Solution

3.2 Timetabling Constraints

3.3 Cost Function Design

3.4 Benchmarking and Validation Framework

3.5 Simulated Annealing for Timetabling

Chapter 4: Results and Discussion

4.1 Experimental Setup and Evaluation Criteria

4.2 Results on the Travelling Salesman Problem

4.3 Results on the Timetabling Problem

4.4 Comparative Discussion

4.5 Limitations

Chapter 5: Conclusion and Future Work

5.1 Conclusion

5.2 Future Work

5.3 Self-Evaluation

Appendix A: Professional Issues

Appendix B: Environment and Execution

Bibliography

Abstract

University timetabling presents a complex optimisation challenge due to the simultaneous presences of strict feasibility requirements and competing quality objectives. As problem size and constraint interaction increase, exact solution methods quickly become infeasible, motivating the use of heuristic and metaheuristic algorithms. This project explores the application of metaheuristic optimisation to university timetabling, with an emphasis on understanding algorithmic behaviour and suitability rather than achieving absolute optimality.

To provide a controlled basis for comparison, several metaheuristic approaches are first examined on the Travelling Salesman Problem, a well-known combinatorial optimisation benchmark. The preliminary analysis highlights differences in robustness, convergence behaviour and representational compatibility across algorithms, informing the selection of an appropriate optimisation strategy for more constrained scheduling problems.

The timetabling problem is modelled using a discrete assignment-based representation that supports local neighbourhood modifications and efficient constraint evaluation. Hard feasibility conditions and softer qualitative preferences are incorporated into a unified objective through a weighted penalty-based cost function, allowing the optimisation process to explore infeasible intermediate states while prioritising the restoration of feasibility.

Results obtained on synthetic timetabling instances indicate that Simulated Annealing is effective in navigating highly constrained search spaces, successfully resolving feasibility violations and improving timetable quality. The findings suggest that careful problem modelling and neighbourhood-based optimisation can yield flexible and interpretable solutions for constrained scheduling problems and provide a foundation for future extensions to larger and more realistic timetabling settings.

Personal Motivation and Future Plans

The project was motivated by an interest in mathematical abstraction and modelling also by a desire to understand how mathematics underpins optimisation problems. Although this work strictly falls under computer science, the work emphasises abstraction, logical structure and analytic reasoning, all of which align closely with my academic interests in pure mathematics. The skills developed through this project in problem formulation, critical analysis and independent investigation provide a strong foundation for my forthcoming Master's degree in mathematics and my aspiration of pursuing doctoral study and an academic career.

Theoretical Background and Useful Definitions

Combinatorial Optimisation problems

Combinatorial optimisation problems arise in settings where the objective is to select the best solution from a finite but technically very large set of possible configurations. These problems are characterised by discrete decision variables and complex constraint interactions, often leading to search spaces that grow exponentially with problem size (Papadimitriou & Steiglitz, 1998). As a result, many such problems are computationally intractable to solve exactly for realistic instance sizes.

Both the Travelling Salesman Problem and university timetabling fall into this class of problems. In each case, a solution corresponds to a specific arrangement of discrete elements (cities in a tour or events in a timetable) and the quality of a solution is evaluated according to an objective function subject to constraints (Burke et al., 2004)

NP-Hardness and Exact Methods

The Travelling Salesman Problem is a classical NP-hard problem, meaning that no polynomial time algorithm is known to solve all instances optimally (Garey & Johnson, 1979). Exact methods, such as brute-force enumeration or dynamic programming, guarantee optimal solutions but are only feasible for very small instances due to factorial or exponential growth in runtime.

University timetabling problems are also NP-hard and can be reduced from other well-known scheduling and graph colouring problems (Even et al., 1976; Schaerf, 1999). As a consequence, exact approaches quickly become impractical once realistic constraints and instance sizes are considered.

Heuristic and Metaheuristic

Heuristic methods are problem specific strategies designed to efficiently find good solutions without guaranteeing optimality. A simple example is the nearest neighbour heuristic for the Travelling Salesman Problem, which greedily constructs a tour by repeatedly selecting the closest unvisited city. While computationally efficient, such heuristics are prone to becoming trapped in poor-quality local minima (Lawler et al., 1985).

Metaheuristics provide higher-level frameworks that guide the search process and are designed to be broadly applicable across different problem domains. They are typically stochastic and iterative, balancing exploration of the search space with exploitation of promising regions. Well-known metaheuristics include Simulated Annealing, Genetic Algorithms and Particle Swarm Optimisation (Glover and Kochenberger, 2003).

Local Search and Neighbourhood Structures

Local search techniques form the basis of many metaheuristic algorithms. Starting from an initial solution, the algorithm repeatedly applies small modifications (known as neighbourhood moves) to

explore nearby solutions. The definition of neighbourhood structure plays a critical role in determining the effectiveness of the search (Aarts & Lenstra, 1997).

In the Travelling Salesman Problem, common neighbourhoods include swap and 2-opt moves, which alter the ordering of cities in a tour. In timetabling problems, neighbourhoods often involve reassigning events to alternative timeslots or rooms. The choice of neighbourhood affects both the ability to improve solution quality and the algorithm's capacity to repair constraint violations.

Simulated Annealing

Simulated Annealing is a probabilistic metaheuristic inspired by the physical process of annealing in metallurgy, where controlled cooling allows a material to reach a low-energy crystalline state. The algorithm was first introduced by Kirkpatrick, Gelatt and Vecchi (1983).

At each iteration, Simulated Annealing considers a neighbourhood solution. Improving moves are always accepted, while worsening moves may be accepted with a probability that depends on the increase in cost and a temperature parameter. As the temperature decreases according to the cooling schedule, the probability of accepting worse solutions reduces, allowing the search to gradually focus on exploitation rather than exploration (Kirkpatrick et al., 1983).

This mechanism enables SA to escape local minima and makes it particularly effective for highly constrained optimisation problems, including timetabling, where temporary infeasibility can facilitate progress towards better solutions (Černý, 1985)

Constraint Handling and Penalty Methods

Constraint handling is a central challenge in constrained optimisation. One widely used approach is the application of penalty functions, where constraint violations are incorporated into the objective function through additional penalty terms (Coello Coello, 2002).

A common distinction is made between hard constraints, which must ultimately be satisfied and soft constraints which represent preferences or quality measures. Hard constraints are assigned dominant penalties to ensure feasibility is prioritised, while soft constraints contribute smaller penalties that guide the optimisation process once feasibility has been achieved (Burke et al., 2004)

Penalty-based methods are particularly well-suited to metaheuristic algorithms such as Simulated Annealing, as they allow the search to traverse infeasible intermediate solutions while still converging towards feasible, high-quality outcomes.

From Benchmarking Problems to Timetabling

Benchmarking problems such as the Travelling Salesman Problem are frequently used to analyse and compare optimisation algorithms due to their simplicity and well understood structure. Insights gained from these benchmarks (such as the impact of neighbourhood design and parameter tuning) can then be transferred to more complex real-world problems (Johnson & McGeoch, 2002).

In this project, the Travelling Salesman Problem serves as a controlled experimental setting in which optimisation techniques are explored before being applied to the richer and more constrained university timetabling problem. This progression allows theoretical ideas to be examined in isolation prior to their integration into a more realistic modelling framework.

Chapter 1: Prep-work and Baseline Algorithms

1.1 Preliminary work

My initial work for this project has focused on building the necessary theoretical and practical foundations for later work in timetabling. During the first week I carried out a review of literature relating to the combinatorial optimisation and metaheuristic algorithms, with particular emphasis on simulated annealing (SA), Particle Swarm Algorithms (PSO) and Genetic Algorithms (GA). Through this review, I developed a deeper understanding of how metaheuristics and how they are well suited to NP-hard problems and how their performance is shaped by representation, neighbourhood structure and parameter choices.

I also used this time to study the Travelling Salesman Problem (TSP) as a classical benchmark in optimisation research. The TSP provides a controlled environment in which the behaviour of different algorithms can be explored before moving on to more complex and constraint-heavy timetabling environment. The TSP is frequently used to validate or compare optimisation methods prior to their adaptation for scheduling tasks.

1.2 Brute Force

I implemented a brute force solver for the TSP, designed to handle small instances of up to ten cities. This implementation served two purposes. First, it provided optimal baseline results against which I can later evaluate and compare to heuristic and metaheuristic approaches. Second it offered an early opportunity to establish coding structure, test distance representations and measure performance characteristics.

I implemented this brute force solver by computing all possible Hamiltonian cycles (with symmetry reduction), evaluating route length using a pre-computed Euclidean distance matrix, then reports the shortest tour.

The benchmark tests demonstrated the expected factorial growth in runtime. The benchmark showed computation times remained extremely small for instances of six and seven cities but increased noticeably as the instance size approached ten cities. Although the absolute runtimes were still low, the growth trend highlights how rapidly exhaustive searches become infeasible, reinforcing the motivation for using heuristic and metaheuristic techniques for larger problems (Garey & Johnson, 1979).

```
--- Brute-Force TSP Benchmark Results ---  
  
6 cities:  
Best route: (0, 1, 3, 5, 2, 4)  
Best distance: 2.4678  
Runtime: 0.0003 seconds  
  
7 cities:  
Best route: (0, 2, 4, 1, 3, 6, 5)  
Best distance: 2.0775  
Runtime: 0.0005 seconds  
  
8 cities:  
Best route: (0, 1, 3, 6, 7, 2, 4, 5)  
Best distance: 2.2706  
Runtime: 0.0039 seconds  
  
9 cities:  
Best route: (0, 2, 5, 1, 6, 8, 3, 4, 7)  
Best distance: 2.5990  
Runtime: 0.0254 seconds  
  
10 cities:  
Best route: (0, 4, 3, 2, 7, 9, 1, 8, 6, 5)  
Best distance: 3.0544  
Runtime: 0.1775 seconds
```

1.3 Nearest neighbours

Next, I used a Nearest Neighbour (NN) approach to tackle the TSP. This algorithm finds its next node by repeatedly selecting the closest unvisited city, producing a route in quadratic time.

This method I used to benchmark performance on problem sizes of between 20 and 200 cities. The runtime performance was excellent, with the 200 city problem taking only 0.0081 seconds. The corresponding tour lengths increased from 4.2034 for 20 cities to 13.3568 for 200 cities. This reflects the growth in Euclidean TSP instances.

In comparison to the brute force results obtained prior, which produced optimal distances between 2.0775 and 3.0544 for 6-10 cities, the NN heuristic trades this for speed. While NN cannot be directly compared in this example due to the scaling of cities I fed to the program, the results clearly show for large instances it has strengths and limitations: it is able to handle extremely large instances but does not necessarily guarantee optimal solutions. (Lawler et al., 1985)

This confirms NN is a suitable baseline heuristic but also emphasises the need for more efficient optimisation methods such as Simulated Annealing, Particle Swarm Optimisation and Genetic Algorithms.

```
--- Nearest Neighbour (NN) TSP Benchmark ---  
  
20 cities:  
  NN distance: 4.2034  
  Runtime: 0.0001 seconds  
  
50 cities:  
  NN distance: 6.5557  
  Runtime: 0.0006 seconds  
  
100 cities:  
  NN distance: 9.3941  
  Runtime: 0.0025 seconds  
  
200 cities:  
  NN distance: 13.3568  
  Runtime: 0.0081 seconds
```

Chapter 2: Metaheuristics on the Travelling Salesman Problem

This chapter introduces the metaheuristic algorithms that form the core of the project. Metaheuristics offer a more powerful alternative to exhaustive and greedy methods by balancing exploration and exploitation. This chapter focusses on three methods: Simulated Annealing (SA), Particle Swarm Optimisation (PSO) and Genetic Algorithms. In this chapter I will be applying them to the TSP to understand their behaviour before adapting them to timetabling.

2.1 Simulated Annealing

The inspiration for the method is taken from the physical process of annealing in metallurgy, where a material is heated and then slowly cooled to reach a stable low-energy state. In terms of optimisation: the temperature represents the probability of accepting worse solutions and the cooling schedule controls the rate at which the randomness decreases.

The algorithm works by starting with a full random tour, visiting every city. Then a small change is made to the route to see if it leads to a shorter tour. If the new route is better, it always accepts it. If the new route is worse, it will accept depending on the temperature value. The higher the temperature, the more likely it is to accept a worse move. As the algorithm runs the temperature slowly decreases. Meaning, at the start it is explorative and tries different routes. By the end it is focused and only keeps improvements. By the end the algorithm behaves like a greedy method, only accepting improvements. When the temperature reaches a very small value, the algorithm stops.

In my code I have used a 2-opt swap method for adjusting the route. This works by picking two cities, reversing a section of the route between and create a new route. A major benefit of this method is it removes crossed edges. This makes the algorithm particularly effective, even in isolation (Johnson & McGeoch, 2002).

```
def two_opt_swap(route):
    n = len(route)
    i, j = sorted(random.sample(range(n), 2))
    new_route = route[:i] + route[i:j][::-1] + route[j:]
    return new_route
```

To implement my cooling method, I used a geometric solution to reduce the temperature. After evaluating a candidate route and deciding whether to accept it, the temperature is updated with a single line:

```
T *= alpha
```

The value of alpha is a constant just less than one (set to my 0.995 in my algorithm), meaning the temperature decreases by the same proportion each time.

I ran the same benchmarks as I did with the nearest neighbours algorithm for a direct comparison. Each individual runtime came in at less than 0.04 seconds. This confirms that my code is efficient and that the overhead of the repeated 2-opt neighbourhood evaluations is manageable even for moderately large problem sizes. However, the solution quality of SA varied significantly. For the 20 cities benchmark algorithm produced a tour marginally shorter than my implementation of the nearest neighbours heuristic. Which indicates the stochastic exploration in SA functions effectively

on small instances. For 50, 100 and 200 cities, these distances substantially larger than those used by the NN algorithm.

```
--- Simulated Annealing (SA) TSP Benchmark ---
```

```
20 cities:
  SA distance: 4.4821
  Runtime:    0.0071 seconds

50 cities:
  SA distance: 7.4666
  Runtime:    0.0106 seconds

100 cities:
  SA distance: 14.4735
  Runtime:    0.0173 seconds

200 cities:
  SA distance: 36.3034
  Runtime:    0.0308 seconds
```

These results suggest either my algorithm cooled too quickly or failed to escape early poor-quality regions. This behaviour is consistent with SA configurations that use an overly aggressive or an insufficiently large initial temperature (Kirkpatrick et al., 1983)

To test this, I increased my starting temperature to 5 to allow early exploration and acceptance of jumping out of bad regions. I also adjusted my alpha (cooling rate) to 0.9992 and doubled my max iterations to give the program its best chance. This resulted in the following benchmarks:

```
--- Simulated Annealing (SA) TSP Benchmark ---
```

```
20 cities:
  SA distance: 4.3494
  Runtime:    0.0392 seconds

50 cities:
  SA distance: 5.5460
  Runtime:    0.0547 seconds

100 cities:
  SA distance: 8.5084
  Runtime:    0.0946 seconds

200 cities:
  SA distance: 17.6330
  Runtime:    0.1768 seconds
```

These clearly show significant improvements from the previous configuration. The benchmarks for 50 and 100 cities now outperform that of the NN algorithm, proving my theory as to why they were not performing as I would have expected.

It is however important to note that the performance for 20 cities decreased this is as we fed the algorithm such a small search space and the NN algorithm already gets so close to optimal for such a small sample. So given the randomness of SA, we can end up with a worse performing benchmark.

As a result, it is clear the configuration and fine tuning has a great impact on the quality of the final route. This will be something I will bear in mind when creating my timetabling solution.

2.2 Particle Swarm Optimisation

This algorithm is a population-based metaheuristic. It is inspired by swarm behaviour in nature, such as the flocking of birds. Unlike SA, which operates on a single solution, Particle Swarm Optimisation (PSO) maintains a swarm of candidate solutions (“particles”) that explore the search space in cooperation.

Because the TSP routes are permutations, we cannot add velocities like in continuous PSO. Instead, each particles movement is represented as a set of swaps that change the order of cities in a route.

These swaps are chosen so that the particle becomes more similar to its own best route and the swarm's best route. By applying only a small number of these swaps at each step, the particle gradually moves toward better solutions while still keeping a valid TSP tour. This allows PSO to behave in its usual cooperative way, but in a form that works for permutation-based problems like the TSP.

My implementation of the PSO did not perform as well as I had expected. Across all benchmark sizes, PSO consistently produced tours that were substantially longer than those obtained by NN and SA. For example, the 50 city instance yielded a PSO distance of 21.8768 compared to 5.5460 and 6.5557 from SA and NN respectively. This gap only widened for the 100 and 200 city cases. This behaviour suggests that the discrete adaptation of PSO I have used here was not effective at preserving good route structure or converging towards high quality solutions (Glover & Kochenberger, 2003).

```
--- PSO TSP Benchmark ---  
  
20 cities:  
  PSO distance: 7.9304  
  Runtime:      0.0536 seconds  
  
50 cities:  
  PSO distance: 21.8768  
  Runtime:      0.0980 seconds  
  
100 cities:  
  PSO distance: 40.8583  
  Runtime:      0.1809 seconds  
  
200 cities:  
  PSO distance: 85.7350  
  Runtime:      0.3404 seconds
```

My theory as to what caused this was the swap-based velocity update likely introduced too much disruption at each iteration. This would indicate PSO is powerful in optimisation however applying it directly to permutation-based problems such as the TSP requires more sophisticated adaptations and a hybrid method will serve it well. An idea I will trial later will be to attempt a 2-opt local search inside each particle.

2.3 Genetic Algorithm

Genetic Algorithms (GA) are population-based metaheuristics inspired by natural evolution. They work by repeatedly improving a population of candidate solutions. This is done by using processes that mirror biological natural selection, crossover and mutation. GAs are especially popular with permutation problems such as the TSP as they naturally retain a diverse set of solutions and combine information from multiple candidates in a structured way.

My Genetic Algorithm implementation represents each candidate solution as a permutation of city indices, corresponding to a valid Travelling Salesman tour. A Euclidean distance matrix is precomputed and used to evaluate the fitness of each individual candidate solution, represented by a complete TSP tour. The algorithm begins by generating an initial population of random tours, ensuring sufficient population variation at the start of the search.

At each generation the search is evaluated by fitness. Selection is performed using a truncation strategy, where the better performing half of the population is retained as parents. New offspring are generated by repeatedly selecting pairs of parents at random and applying Ordered Crossover (OX). This crossover operator copies a contiguous segment of from one parent and fills the remaining positions using the city order from the other parent, ensuring that the offspring remain valid permutations while inheriting structural information from both parents.

To maintain population variation and reduce the risk of premature convergence, a swap mutation operator is applied to offspring of low probability. This mutation randomly exchanges the positions

of two cities in the tour, introducing the small modifications without invalidating the solution. The offspring population fully replaces the previous generation and this evolutionary process is repeated for a fixed number of generations. After completion, the best individual in the final population is returned as the algorithm's solution.

From running this implementation on my benchmark, I found GA performed poorly producing longer tours than both the Nearest Neighbours and Simulated Annealing algorithms.

```

-- GA TSP Benchmark --

20 cities:
GA distance: 5.6912
Runtime: 0.1370 seconds

50 cities:
GA distance: 18.2728
Runtime: 0.4126 seconds

100 cities:
GA distance: 42.3482
Runtime: 1.3830 seconds

200 cities:
GA distance: 89.2436
Runtime: 4.6152 seconds

```

Solution quality degraded rapidly as the number of cities increased, with particular weak performances on the 100 and 200 city benchmarks. In addition, the GA produced significantly higher runtimes due to its population-based structure, reaching several seconds for the largest problem size without delivering corresponding improvements in solution quality. This behaviour can be attributed to the disruptive effects crossover, mutation and absence of an embedded local search mechanism, which limited the algorithm's ability to preserve and refine high-quality route structure. These results indicate that, in its basic form, the Genetic algorithm is not well suited to solving the TSP at larger scales (Lawler et al., 1985).

2.4 Combined analysis of Metaheuristics on TSP

This section compares the performance of the metaheuristic approaches implemented in this project for the Travelling Salesman Problem. The comparison focusses on three key criteria: solution quality, computational efficiency and scalability.

Simulated Annealing achieved the best overall performance. After tuning, it improved upon the Nearest Neighbours heuristic for medium sized instances while remaining computationally efficient. Its ability to escape local minima and iteratively refine solutions makes it the most suitable algorithm for further adaptation and hybridisation. In contrast, Particle Swarm Optimisation and the Genetic Algorithm performed poorly across all instances. Both methods produced significantly longer tours while also incurring higher computational cost, indicating that their discrete adaptations were insufficient for the TSP without hybridisation or local search enhancements.

Algorithm	Solution Quality	Runtime	Scalability	Overall Assessment
Simulated Annealing	Best Overall (after tuning)	Low-moderate	Good	Most effective metaheuristic
Particle Swarm Optimisation	Poor	Moderate-high	Poor	Ineffective in this form

Genetic Algorithm	Poor	High	Poor	Inefficient without hybridisation
-------------------	------	------	------	-----------------------------------

Chapter 3: Adapting Metaheuristics to the timetabling problem

3.1 Timetabling problem representation

3.1.1 Design choices

The timetabling representation was designed to support efficient neighbourhood-based optimisation while remaining flexible enough to model the multiple constraints that appear in real-world scheduling. Unlike the Travelling Salesman Problem, which can be represented as a single permutation, timetabling requires simultaneous assignment decisions. For this reason, the representation focusses on assigning each event to a specific timeslot and room, allowing conflicts involving students, lecturers and rooms to be explicitly modelled.

A key design choice was to prioritise fast constraint evaluation, as Simulated Annealing requires the assessment of a large number of candidate solutions. To achieve this, auxiliary data structures are maintained that track room usage, lecturer assignments and cohort attendance across timeslots. This enables hard constraint violations to be detected in constant time, significantly improving efficiency during optimisation. Rather than enforcing feasibility directly, constraint violations are penalised through the cost function, allowing the search to move through infeasible intermediate solutions and escape poor local minima.

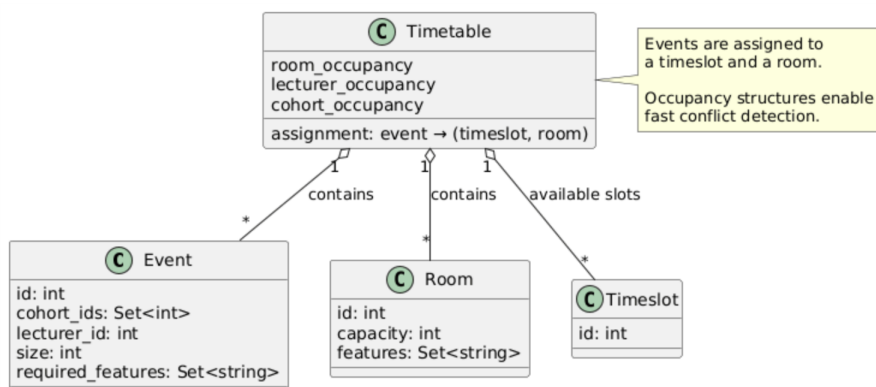
3.1.2 Coding a solution

A timetable is stored as a mapping from identifiers to pairs of timeslots and rooms, representing the current schedule. Each event, room and timeslot are defined explicitly, with events containing information such as associated student cohorts, lecturer identity, class size and rooms specifying capacity constraints.

Alongside the primary assignment mapping, additional lookup structures record which rooms, lecturers and cohorts are occupied each timeslot. These structures are updated whenever an event is placed or moved and allow clashes to be detected efficiently without scanning the entire timetable.

This design makes it straightforward to implement local modifications, such as moving an event to a different timeslot or swapping two events. The timetable can also be copied efficiently, enabling candidate solutions to be evaluated and either accepted or rejected during the optimisation process.

The following figure illustrates the structure of the timetabling model:



3.2 Timetabling Constraints

The timetabling problem is defined using two categories of constraints:

Hard Constraints

These constraints must always be satisfied. Any timetable that violates any hard constraint is infeasible.

Typical examples of hard constraints include:

- A student cannot be assigned to more than one event in the same timeslot
- A teacher cannot be assigned to more than one event in the same timeslot
- A room cannot host more than one event at a time
- Room capacity must be sufficient for the event

Hard constraints define the feasibility in the timetabling problem and represent conditions that must not be violated in any acceptable timetable (Burke et al., 2004). In this model, hard constraints include room clashes, lecturer clashes, cohort clashes, room capacity violations, feature mismatches and unassigned events. These constraints are evaluated independently of the optimisation process to ensure correctness and transparency.

A dedicated hard constraint evaluation function was implemented to count violations directly from the timetable assignment. Rather than relying solely on occupancy structures, violations are derived from the assignments, ensuring robustness even when multiple events conflict within the same slot. For each constraint type, the total number of violations is recorded and aggregated into a single violation count. This design allows the optimisation process to distinguish between feasible and infeasible timetables while also identifying which constraints are most frequently violated.

By separating hard constraint evaluation from the optimisation logic, the implementation remains modular and adaptable. Additional hard constraints can be introduced without modifying the core algorithm and constraint statistics can be reported independently for analysis and debugging purposes.

Soft Constraints

These represent preferences rather than absolute requirements.

Typical examples of soft constraints include:

- Minimising gaps in students' timetables
- Avoiding early morning or late evening timeslots
- Distributing events evenly across the week
- Allocating preferred rooms where possible

Unlike hard constraints, soft constraints do not determine feasibility and are therefore not enforced strictly. Instead, violations of soft constraints are incorporated into the objective function as weighted penalty terms, allowing the optimisation process to balance timetable quality against feasibility. This approach enables the search to explore a wide range of candidate solutions while gradually improving desirable properties such as schedule compactness and the distribution of

events across the day. The specific weighting and treatment of these penalties is discussed in the following section.

3.3 Cost Function Design

Building on the representation and constraint definitions introduced in the previous sections, the timetabling problem is formulated as a single-objective optimisation task by means of a weighted cost function. Rather than enforcing feasibility directly, the cost function translates both hard and soft constraint violations into numerical penalties, allowing candidate timetables to be compared and iteratively improved using Simulated Annealing.

The total cost is defined as:

$$Cost(T) = W_h V_h(T) + \sum_{i=1}^k W_i V_i(T)$$

Where:

- T - is a candidate timetable
- $V_h(T)$ - is the number of hard constraint violations
- W_h - is the hard-violation penalty weight
- $V_i(T)$ - is the number of soft constraint violations
- W_i - are the corresponding soft weights

Rather than using a single trade-off parameter, each constraint is weighted individually, allowing finer control over the optimisation.

Hard constraints are incorporated as heavily weighted penalty terms to ensure that feasibility is strongly prioritised during optimisation. Any violation of a hard constraint contributes to a large increase in the total cost, making infeasible solutions highly undesirable but not entirely inaccessible. This design choice allows the search to pass temporarily through infeasible intermediate states, which can be beneficial for escaping poor local minima, while still ensuring convergence towards feasible timetables (Coello Coello, 2002; Burke et al., 2004).

Once hard constraint violations have been reduced, softer measures of timetable quality become more influential. Soft constraint penalties are therefore assigned significantly smaller weights and are used to guide the optimisation towards more desirable scheduling patterns. In this project, these penalties focus on reducing gaps in the student schedules and discouraging the use of particularly early and late timeslots, reflecting common practical considerations in academic timetabling.

The cost function implemented to return both a scalar objective value and when required a detailed breakdown of individual penalty contributions. This breakdown is used during benchmarking and validation to confirm that each component of the objective behaves as expected and contributes appropriately to the total cost. By structuring the objective function this way, the optimisation process naturally transitions from resolving feasibility issues to improving the overall timetable quality as the search progresses.

3.4 Benchmarking and Validation Framework

Prior to applying Simulated Annealing, a small benchmarking framework was implemented to validate the timetabling representation, constraint evaluation and cost function. Synthetic timetabling instances were constructed consisting of a limited number of events, rooms and timeslots with overlapping cohorts and lecturers to ensure the presence of natural conflicts.

For each instance, an initial timetable was generated using random assignment and evaluated using the proposed cost function. A detailed breakdown of hard and soft constraint penalties was examined to confirm that infeasible timetables incurred large penalties and that soft penalties reflected qualitative scheduling properties.

Additional manual modifications to event assignments were performed to verify that small changes in the timetable produced corresponding and interpretable changes in the objective value. These checks confirmed that the cost function provided a meaningful optimisation signal and that the timetabling model behaved as expected.

3.5 Simulated Annealing for Timetabling

On a small synthetic timetabling instance, the initial timetable was feasible, with no hard constraint violations but incurred a non-zero quality cost due to gaps in the student schedules and the use of late timeslots. The initial cost was 19, reflecting the penalties from the soft constraints.

Simulated Annealing was applied using the neighbourhood moves and cost function described previously. Over the course of the optimisation, the objective value was consistently reduced and a timetable with zero total cost was obtained. This improvement was achieved by eliminating both gap and late-slot penalties while maintaining feasibility throughout the search.

The table that follows summarises the constraint violations and cost before and after the application of Simulated Annealing.

Stage	Hard violations	Soft gaps	Soft late	Total
Initial	0	15	4	19
After SA	0	0	0	0

These results demonstrate that, even on a simple instance, Simulated Annealing can effectively improve timetable quality once feasibility has been achieved, validating its suitability for the proposed timetabling formulation. Although the initial timetable was already feasible, the experiment illustrates how the optimisation process can focus on improving the qualitative scheduling properties once hard constraints have been satisfied.

To further assess the behaviour of the optimisation process, a second more constrained synthetic timetabling instance was considered. In this instance, the number of available rooms was reduced, increasing competition for resources and resulting in an initially infeasible timetable with multiple hard constraint violations, including clashes and resource violations.

When Simulated Annealing was applied to this harder instance, the optimisation process fully repaired all hard constraint violations, reducing the hard violation count from two to zero and restoring feasibility. Once feasibility had been restored, the algorithm continued to improve

timetable quality by reducing soft constraint penalties associated with gaps in student schedules and the use of late timeslots.

These results demonstrate that Simulated Annealing is not only effective at improving timetable quality when feasibility is already satisfied but is also capable of repairing infeasible initial solutions by navigating through intermediate states with temporary constraint violations. This behaviour directly supports the design choice of incorporating hard constraints as penalised terms rather than always enforcing feasibility strictly.

Stage	Hard violations	Soft penalties	Total cost
Initial	2	17	2,000,017
After SA	0	0	0

Chapter 4: Results and Discussion

4.1 Experimental Setup and Evaluation Criteria

This section evaluates the performance and behaviour of the implemented metaheuristic algorithms on both the Travelling Salesman Problem (TSP) and the university timetabling problem. The aim is not to perform large-scale benchmarking, but rather to gain comparative insight into how different optimisation approaches behave on combinatorial optimisation problems of increasing complexity and to assess their suitability for constrained scheduling tasks.

All experiments were conducted on synthetic problem instances to allow controlled testing and clear interpretation of results. For the TSP, randomly generated Euclidean instances of varying sizes were used, enabling the direct comparisons between Simulated Annealing, Particle Swarm Optimisation and Genetic Algorithms under consistent conditions. For the timetabling problem small synthetic instances were constructed to reflect realistic scheduling constraints, including limited room availability, shared cohorts and lecturer assignments.

Algorithm performance was evaluated primarily using solution quality and constraint satisfaction. For the TSP, solution quality was measured by total tour length produced by each algorithm, with shorter tours indicating better performance. For the timetabling problem, evaluation was based on a weighted cost function incorporating both hard and soft constraints. Hard constraint violations were heavily penalised to prioritise feasibility, while soft constraints contributed smaller penalties reflecting timetable quality. Lower total cost therefore corresponds to more feasible and higher quality timetables.

4.2 Results on the Travelling Salesman Problem

The Travelling Salesman Problem was used as an initial testbed to compare the behaviour of different metaheuristic optimisation techniques in a controlled setting. As a well-studied combinatorial optimisation problem, TSP allows algorithmic strengths and weaknesses to be observed clearly before transitioning to more complex and constrained timetabling domain.

Simulated Annealing demonstrated consistently strong performance on the TSP across a range of instance sizes. The algorithm reliably produced high-quality tours and demonstrated stable convergence behaviour. Its effectiveness can be attributed to the use of simple but powerful neighbourhood moves, such as 2-opt swaps, combined with a cooling schedule that allows early exploration of the search space while gradually focussing on exploitation. Simulated Annealing showed robustness to initial solutions and maintained reasonable computational efficiency as problem size increased.

The Genetic Algorithm achieved acceptable solution quality on smaller and medium sized TSP instances but generally required longer runtimes to converge. While crossover operators enabled the combination of useful structural features from parent tours, they also introduced disruptive changes that occasionally degraded solution quality before improvement occurred. The algorithm's performance was sensitive to parameter choices such as population size, mutation rate and number of generations making consistent tuning more challenging.

Particle Swarm Optimisation performed poorly on the TSP relative to the other approaches. Despite adaptations to operate on permutations, the algorithm struggled to effectively explore the discrete search space, leading to significantly worse solution quality. This reflects a fundamental mismatch between PSO's continuous optimisation framework and the permutation-based nature of the TSP.

As a result of this behaviour, Particle Swarm Optimisation was not considered further for the timetabling problem.

Overall, the TSP experiments indicated that Simulated Annealing offered the most reliable balance between solution quality, stability and conceptual simplicity. These findings motivated its selection as the primary optimisation technique for the timetabling problem.

4.3 Results on the Timetabling Problem

The timetabling problem introduces a substantially higher level of complexity than the TSP due to the presence of multiple hard constraints that must be satisfied to achieve feasibility. Unlike the TSP, where all candidate solutions are valid tours, many timetabling solutions are infeasible, making constraint handling a central challenge.

On an initial synthetic timetabling instance that was feasible but of low quality, Simulated Annealing successfully improved the timetable by eliminating soft constraint penalties. Starting from a timetable with no hard violations but a non-zero cost due to gaps in student schedules and the use of late timeslots, the optimisation process reduced the total cost to zero. This demonstrated the algorithm's ability to improve timetable quality while preserving feasibility once a valid solution had been found.

To further assess optimisation behaviour, a more constrained timetabling instance was considered in which resource availability was reduced, leading to an initially infeasible timetable with multiple hard constraint violations. When Simulated Annealing applied to this harder instance, the algorithm was able to substantially reduce hard constraint violations and in several runs restore full feasibility. After feasibility was achieved, further optimisation focused on reducing soft penalties, improving the overall structure and quality of the timetable.

Across both instances, a consistent pattern was observed: Simulated Annealing first prioritised the reduction of hard constraint violations due to their dominant contribution to the cost function and then shifted towards improving qualitative aspects of the timetable once feasibility had been established. This behaviour validates the penalty-based cost function design and demonstrates the suitability of Simulated Annealing for constrained scheduling problems. (Černý, 1985; Kirkpatrick et al., 1983)

4.4 Comparative Discussion

The comparative behaviour of the optimisation methods highlights the importance of aligning algorithmic assumptions with problem structure. Although all three metaheuristics are general-purpose optimisation techniques, their effectiveness varied substantially depending on how naturally their underlying mechanisms mapped onto the discrete and highly constrained nature of the problems considered.

A key factor influencing performance was the degree to which each algorithm could exploit local structural information. Simulated Annealing operates directly on small, interpretable neighbourhood moves, enabling incremental improvements that respect the combinatorial structure of both the Travelling Salesman Problem and the timetabling problem. This locality proved especially valuable in the timetabling context, where resolving a single hard constraint violation often requires only a small, targeted modification rather than a global restructuring of the solution.

In contrast, Genetic Algorithms exhibited a tendency to disrupt partially feasible structures during crossover. While this behaviour can be beneficial in unconstrained or weakly constrained problems by promoting exploration, it becomes detrimental in heavily constrained settings such as

timetabling, where many combinations of events, rooms and timeslots are invalid. As a result, much of the search effort is spent rediscovering feasibility rather than refining solution quality, limiting the algorithm's effectiveness in this domain.

The limitations of Particle Swarm Optimisation further reinforce this observation. PSO's reliance on continuous position updates and velocity vectors introduces a conceptual mismatch when applied to permutation-based problems. Although discrete adaptations exist, they often obscure the intuitive meaning of the algorithm's parameters and reduce transparency in how improvements are achieved. In the context of this project, such adaptations offered little practical advantage over simpler neighbourhood-based methods.

An additional insight emerges from the penalty-based formulation of the timetabling objective function. Allowing infeasible intermediate solutions, combined with a strong separation between hard and soft penalty weights, enables the optimisation process to prioritise feasibility repair before addressing qualitative scheduling preferences. Simulated Annealing's probabilistic acceptance mechanism is particularly well suited to this strategy, as it permits controlled traversal through infeasible regions of the search space without becoming trapped in poor local minima.

Taken together these observations suggest that the success of Simulated Annealing in this project is not solely due to parameter choices or implementation details but rather to a fundamental compatibility between the algorithm's local search dynamics and the structural properties of constrained scheduling problems. This reinforces the broader principle that, in combinatorial optimisation, algorithm selection should be guided as much by problem representation and constraint structure as by raw optimisation power.

4.5 Limitations

While the results presented demonstrate the effectiveness of Simulated Annealing for both the Travelling Salesman and timetabling problems, several limitations of the study should be acknowledged. These limitations primarily relate to problem scale, data realism and algorithm configuration and provide important context for interpreting the results.

First, the experiments were conducted on relatively small, synthetic problem instances. For the TSP, instance sizes were chosen to allow multiple algorithmic comparisons within reasonable computational time. For the timetabling problem, simplified instances were constructed to facilitate controlled evaluation of constraint handling and optimisation behaviour. Although these instances capture key structural characteristics of real-world scheduling problems, they do not reflect the full complexity of large institutional timetables with hundreds of events, rooms and varied constraints. Consequently, the results should be viewed as illustrative rather than representative of performance at industrial scale.

Second, algorithm parameters were selected through manual experimentation rather than systematic tuning. While this was sufficient to demonstrate stable and meaningful optimisation behaviour, alternative parameter choices could lead to improved performance or different convergence characteristics. In particular, the cooling schedule and neighbourhood selection in Simulated Annealing play a significant role in balancing exploration and exploitation. A more exhaustive parameter sensitivity analysis or automated tuning strategy was beyond the scope of this project.

A further limitation is that the optimisation approaches considered operate in a single instance setting. In a practical timetabling environment, schedules are often generated repeatedly across academic years or updated dynamically in response to changes such as room unavailability or enrolment fluctuations. The methods evaluated in this project do not explicitly exploit information from previously solved instances, which may limit their efficiency in such recurring scenarios.

To address some of these limitations reinforcement algorithms such as Q-learning could have been used. By learning policies that map timetable states to scheduling actions through repeated interaction and feedback, Q-learning-based methods can, in principle, adapt over time and reuse knowledge across similar problem instances. This contrasts with Simulated Annealing, which treats each instance independently. However, applying reinforcement learning to timetabling introduces additional challenges, including the design of compact state representations and effective reward functions for highly constrained search spaces. Exploring these trade-offs in greater depth is an area to be explored further as I continue to develop this research.

Despite these limitations, the project successfully demonstrates how penalty-based cost functions and neighbourhood-based metaheuristics can be combined to address constrained scheduling problems in a transparent and interpretable manner. The limitations identified not only contextualise the results but also motivate several clear directions for extending the work toward more realistic and adaptive timetabling systems.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

This project investigated the application of metaheuristic optimisation techniques to combinatorial optimisation problems ultimately focussing on university timetabling. The work progressed from the Travelling Salesman Problem as a controlled testbed to a more complex and constrained timetabling formulation, allowing algorithmic behaviour to be studied across problem domains of increasing difficulty.

Several optimisation methods were implemented and evaluated, including Simulated Annealing, Genetic Algorithms and Particle Swarm optimisation. Initial experiments on the Travelling Salesman Problem demonstrated that Simulated Annealing consistently produced high-quality solutions with stable convergence behaviour, while Genetic Algorithms exhibited slower convergence and greater sensitivity to parameter choices. Particle Swarm Optimisation performed poorly on permutation-based instances, highlighting a mismatch between its continuous optimisation framework and discrete combinatorial problems. These results motivated the selection of Simulated Annealing as the primary optimisation method for the timetabling problem.

The timetabling problem was modelled using a flexible assignment-based representation that supports efficient neighbourhood search and modular constraint evaluation. A penalty-based cost function was designed to incorporate both hard and soft constraints. This formulation allows infeasible intermediate solutions and prioritises feasibility through dominant hard constraint penalties, enabling the optimisation process to repair constraint violations before improving timetabling quality.

Experimental results on synthetic timetabling instances demonstrated that Simulated Annealing is effective in both improving the quality of feasible timetables and repairing infeasible initial solutions. On simpler instances, the algorithm successfully eliminated soft constraint penalties while maintaining feasibility. On more constrained instances Simulated Annealing was able to reduce and in several cases, remove hard constraint violations before further improving timetable quality. These results validate the chosen cost function design and confirm the suitability of Simulated Annealing for constrained scheduling problems.

Beyond empirical performance, the project highlights the importance of aligning optimisation methods with problem structure. Algorithms that operate on local, interpretable neighbourhoods and support controlled exploration of the search space proved more effective than approaches that rely on disruptive global operators or continuous representations. This reinforces the broader principle that successful combinatorial optimisation depends as much on representation and constraint handling as on the choice of optimisation paradigm itself.

5.2 Future Work

Despite the positive results obtained, several limitations of the study suggest clear directions for future work. The experiments were conducted on relatively small, synthetic problem instances and extending the framework to larger, real-world timetabling datasets would allow more comprehensive evaluation and reveal additional challenges related to constraint interaction and computational efficiency.

Future work could also explore more systematic approaches to algorithm configuration. Automated parameter tuning, adaptive cooling schedules or dynamic neighbourhood selection strategies may

further improve optimisation performance and robustness, particularly for larger and more complex timetabling instances.

The timetabling model itself could be extended to incorporate additional realistic constraints, such as lecturer availability preferences, fairness considerations or multi-week scheduling structures. These extensions would increase the practical applicability of the framework while preserving the underlying optimisation approach.

A further promising direction lies in the exploration of learning-based methods for repeated or dynamic scheduling scenarios. Reinforcement learning approaches such as Q-learning, could potentially learn policies that guide scheduling decisions across similar problem instances, complementing the single-instance optimisation performed by Simulated Annealing. Investigating hybrid approaches that combine constructive heuristics, learning-based guidance and neighbourhood-based optimisation represents an interesting avenue for future research.

5.3 Self-Evaluation

Overall, the project progressed successfully and met its original aims, though not without challenges. One of the main strengths of the project was the clear progression from simpler benchmark problems to a more complex and realistic timetabling. Beginning with the Travelling Salesman Problem provided a controlled environment in which to explore different optimisation techniques and understand their behaviour, which proved valuable when designing and applying methods to the timetabling problem.

A particularly successful aspect of the project was the emphasis on problem representation and the design of the cost function. Spending a considerable amount of time early on, defining clear constraints, penalty structures and validation mechanisms helped ensure that later experimentation was of great quality. The decision to prioritise feasibility through dominant hard constraint penalties, while allowing infeasible intermediate solutions, proved effective and aligned well with the chosen optimisation approach. This highlighted the importance of modelling choices in determining the success of an optimisation system.

The project also reinforced the value of modular and incremental development. Breaking the system into distinct components made it easier to test, debug and extend the codebase. This approach helped manage complexity and reduced the risk of errors being left unnoticed. In contrast, earlier attempts to work on multiple components simultaneously were less productive and emphasised the importance of structured planning.

Not all aspects of the project progressed as smoothly as I would have hoped. Parameter selection for the metaheuristic algorithms required iterative experimentation and it was not always clear how changes would affect performance. While this process was time-consuming, it provided valuable insight into the sensitivity of optimisation methods and the limits of empirical tuning. Additionally, the use of synthetic instances, while necessary for evaluation, limited the extent to which results could be generalised to real-world scenarios.

One area for improvement was my use of version control. While Git was used to manage the codebase, it was not used as consistently or effectively as it should have been. In particular, more frequent commits and proper use of branching would have provided a better record of development and easier access to older versions of my codebase. This would have been a great help during experimentation. Reflecting on this, more disciplined use of version control would be a major priority for future projects.

From a personal perspective, the project significantly improved my ability to manage open-ended pieces of work. Balancing implementation, experimentation and writing the report required careful time management and regular assessment of priorities. Encountering unexpected difficulties,

encouraged a more reflective approach to problem-solving, where revisiting assumptions and adapting plans became an integral part of progress rather than a setback.

Perhaps the most important lesson I learned was that successful projects depend as much on clarity of thinking and structure as they do on technical skill. Underlying why a particular approach works and being able to explain its limitations proved more valuable than simply implementing more complex code. This insight has shaped my approach to any future work and I will carry that with me through my academic career.

Appendix A: Professional Issues

As computing increasingly shapes how organisations operate, professional and ethical considerations become more important in software development. This is particularly true for automated scheduling systems, such as timetabling software, where decisions made by algorithms directly affect students, teaching staff and the day-to-day running of an institution. While optimisation techniques can offer clear efficiency benefits, they also raise questions around trust, reliability and fairness within the role of automation.

One issue that became apparent during this project is trust in algorithmic decisions. When a timetable is generated automatically, users may not understand why certain classes are scheduled at inconvenient times or why some staff appear to have less favourable teaching patterns. Even if the timetable is technically valid, a lack of transparency can lead to frustration or perceptions of unfairness. From a professional perspective, this highlights that producing a correct solution is not always sufficient. Users must also understand, at least at a high level, how decisions are being made.

This concern influenced several design choices in this project. The use of a penalty-based cost function and neighbourhood-based optimisation methods such as Simulated Annealing results in behaviour that can be reasoned about intuitively. Improvements to the timetable occur with through small, local changes that reduce clearly defined penalties, such as clashes or gaps. This makes it easier to explain why a particular timetable is preferred over another, which is important when systems are used in settings that affect people directly.

Another important professional issue is reliability and correctness. Timetabling systems are often relied upon as authoritative sources of information and errors can have disruptive consequences. Issues such as room clashes, overlapping lectures or inaccessible schedules can cause confusion and undermine confidence in institutional systems. Although these failures may not pose immediate safety risks, they can still have significant practical and educational impacts. This places a responsibility on developers to ensure that such systems are carefully tested and validated before use.

The benchmarking and validation work carried out in this project reflects this responsibility. Rather than focussing solely on optimisation performance, explicit effort was made to verify constraint handling and feasibility detection. This reinforced the idea that an optimisation system is only useful if it behaves predictably and correctly, particularly in edge cases. From a professional standpoint, prioritising validation helps ensure that systems can be trusted in the real-world.

Fairness also emerged as a key consideration. Optimisation systems inevitably reflect the priorities encoded in their objective functions. In the case of timetabling, these priorities determine which constraints dominate and which aspects of timetable quality are emphasised. If these choices are made without careful thought, there is a risk that the system may consistently disadvantage certain groups. This project highlights how such priorities are expressed through penalty weights and constraint definitions, emphasising that optimisation systems are not value neutral.

This observation has broader ethical implications. Developers have a responsibility to be aware that technical design decisions can embed assumptions and biases into a system. In practical settings, it is therefore important that optimisation objectives are informed by stakeholder input and reviewed by regularly, rather than treated as purely technical parameters.

There is also a wider management and governance issue associated with automated decision-making systems. Because the output of an algorithm can be seen as objective and authoritative, there is a risk that organisations rely on them without any critical thinking. In complex environments such as university scheduling, no algorithm can capture all contextual or human

factors. As a result, automated systems should support human decision-making rather than replace it entirely.

In this project, the optimisation framework is best viewed as a tool to support decision making. It generates candidate timetable based on formal constraints and optimisation criteria, but human judgement remains necessary to assess factors that are difficult to encode computationally. So, recognising the limits of automation is an important aspect of professional responsibility.

Finally, the project highlighted the importance of professional competency and responsibility in development practices. Building an optimisation system that is modular, testable and easy to extend requires careful planning and clear structure from the outset. Systems that lack this structure can become difficult to maintain and may introduce subtle errors as they evolve. By emphasising modular constraint evaluation and structured benchmarking, the system developed in this project remained understandable and robust, reflecting good software engineering practice.

In summary, the development of automated timetabling systems involves more than solving a technical optimisation problem. Issues of trust, reliability, fairness and responsibility are integral to design and implementation choices. This project demonstrates that addressing these issues alongside technical challenges is essential for producing systems that are not only effective but also appropriate for real-world use.

Appendix B: Environment and Execution

All software developed for this project has been implemented in Python and is included with the submitted source code. The project is organised as a modular codebase, with separate components for problem representation, constraint evaluation, cost function design, neighbourhood generation, optimisation algorithms and benchmarking.

Environment

Development and Test Environment:

- Programming language: Python 3.13.5
- Operating system: macOS 26.2
- Python libraries used:
 - The standard Python library
 - NumPy
- Execution method: Command-line interface

Minimum Requirements to Run the Project:

- Programming language: Python 3.9 or later
- Operating system: Any OS that supports Python
- Additional Python libraries required: NumPy
- Hardware:
 - Any modern 64-bit CPU
 - RAM: 4GB minimum, 8GB recommended
 - Storage Approximately 100MB of free space for source code and outputs

How to Run the Benchmarks

All experiments reported in this project can be reproduced using the provided benchmark scripts. The main entry point for the timetabling experiments is the `benchmarks.py` file located within the Timetabling directory.

From the root directory of the project, the benchmarks can be executed using the following command:

Python -m product.Timetabling.benchmarks

The command runs, in sequence:

- The constraint validation suite
- Simulated Annealing applied to a feasible timetable
- Simulated Annealing applied to an infeasible timetable

The output is printed directly to the console and includes feasibility checks, detailed cost breakdowns and before-and-after comparisons that correspond to the results discussed in the report

All benchmarks for the Travelling Salesman Problem can be run directly from the desired file like any other python file.

The submitted codebase is submitted as follows:

```
PROJECT/
|
|-- README.md
|
|-- documents/
|   |-- Project_Final_Report.pdf
|
|-- product/
|   |-- Timetabling/
|   |   |-- __init__.py
|   |   |-- benchmarks.py
|   |   |-- benchmarks_constraints_suite.py
|   |   |-- constraints.py
|   |   |-- cost_function.py
|   |   |-- models.py
|   |   |-- neighbourhoods.py
|   |   |-- simulated_annealing.py
|   |-- TSP/
|   |   |-- TSPBruteForce.py
|   |   |-- TSPNearestNeighbour.py
|   |   |-- TSPSimulatedAnnealing.py
|   |   |-- TSPParticleSwarmOptimisation.py
|   |   |-- TSPGeneticAlgorithm.py
```

A short demonstration video has been provided to show the execution of the software and the resulting output. The video demonstrates running the benchmarks from the timetabling problem. It can be found at this link:

<https://www.youtube.com/watch?v=TSVt5sy5Jy8>

Bibliography

- Aarts, E., & Lenstra, J. K. (1997). *Local Search in Combinatorial Optimization*. Wiley.
- Burke, E. K., De Causmaecker, P., Petrovic, S., & Vanden Berghe, G. (2004). Metaheuristics for handling timetabling problems. In *Practice and Theory of Automated Timetabling IV*. Springer.
- Černý, V. (1985). Thermodynamical approach to the travelling salesman problem. *Journal of Optimization Theory and Applications*, 45(1), 41–51.
- Coello Coello, C. A. (2002). Theoretical and numerical constraint-handling techniques used with evolutionary algorithms. *Computer Methods in Applied Mechanics and Engineering*, 191(11–12), 1245–1287.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman.
- Glover, F., & Kochenberger, G. A. (2003). *Handbook of Metaheuristics*. Springer.
- Johnson, D. S., & McGeoch, L. A. (2002). Experimental analysis of heuristics for the STSP. In *The Travelling Salesman Problem and Its Variations*. Springer.
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
- Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (1985). *The Travelling Salesman Problem*. Wiley.
- Papadimitriou, C. H., & Steiglitz, K. (1998). *Combinatorial Optimization: Algorithms and Complexity*. Dover.
- Schaerf, A. (1999). A survey of automated timetabling. *Artificial Intelligence Review*, 13, 87–127.